Neural Networks in KingCobra: An Expert Guide to std.nn

Written for KingCobra 1.0.0. Every program in this guide was run on the real interpreter, and the output shown is what it printed. Numbers that depend on the random generator use a fixed `nn.seed`.

This guide is for readers who already know neural networks and want to know exactly how `std.nn` works: its conventions, its tensor and linear-algebra toolkit, its layer and training APIs, how to verify a model, and how to build reinforcement learning agents, including **Double DQN**. It assumes you know the KingCobra language itself. If you do not, read the language tutorial first.

It ends with a set of work problems that give the problem and only the final answer.

---

1. Design and conventions

```
import std.nn;
```

`std.nn` is a native, dependency-free deep-learning library: no Python, no GPU, no downloaded weights. Everything is **float64 on one CPU thread**. Its design rules explain most of its behavior:

- **One small interface.** Every layer implements forward, backward and save. Containers (`sequential`, `residual`) nest freely, and the training engine talks only to that interface.
- **Verified gradients.** Every layer, activation and loss is checked against numerical gradients with `nn.gradcheck`.
- **Shapes are batch-first:** `[N, features]`, `[N, C, H, W]`, `[N, T, features]`. Tensors have at most 6 dimensions.

1.1 Conventions you must know

Topic	Rule
Inputs	Anywhere a tensor is wanted you may pass a tensor handle, a number, or a rectangular nested array.
Options	Trailing name/value pairs: `nn.dense(4, 8, "init", "he", "bias", false)`. An unknown option is an error that lists the valid ones.

Topic	Rule
Results	Multi-field results are structs (`h.loss`, `g.max_rel_error`). Ranked results are arrays of `[item, score]`.
Predictions	`predict`, `predict_class`, `to_array` return nested arrays. `forward` and the math functions return **tensor handles**.
Randomness	One generator drives initializers, dropout, shuffling and splits. `nn.seed(n)` makes a run reproducible.
Training vs inference	`fit` trains. `predict` and `evaluate` run in inference mode. `nn.forward(m, x)` defaults to **training mode**: pass `false` as the third argument for inference.

1.2 Memory

KingCobra has no garbage collector, and tensors are native handles. `fit`, `evaluate`, `predict` and `train_step` allocate nothing per step. In a **manual loop** you must call `nn.free(t)` on every per-step tensor (every `forward` output). Models, losses and optimizers live until the program ends. The core `free()` does not apply to tensor handles: use `nn.free`.

Two habits keep long loops flat in memory: free `forward` outputs as soon as you have read them, and prefer `train_step` or `fit` when you do not need to touch the intermediate tensors.

---

2. Tensors and math

2.1 Creation, shape and broadcasting

```
import std.nn;

fn main() {
    let a = nn.tensor([[1, 2, 3], [4, 5, 6]]);
    let b = nn.tensor([10, 20, 30]);
    print(nn.shape(a), nn.add(a, b));           // b broadcasts across the
rows
    print(nn.matmul(a, nn.transpose(a)));
    print(nn.sum(a), nn.sum(a, 0), nn.mean(a, 1), nn.max(a, 1), nn.argmax(a,
1));
    print(nn.reshape(nn.arange(6), [2, -1]));    // one -1 is inferred
}
```

```
[2, 3] tensor[2,3] [[11, 22, 33], [14, 25, 36]]
tensor[2,2] [[14, 32], [32, 77]]
tensor[] 21 tensor[3] [5, 7, 9] tensor[2] [2, 5] tensor[2] [3, 6] tensor[2]
[2, 2]
tensor[2,3] [[0, 1, 2], [3, 4, 5]]
```

Elementwise functions (`add sub mul div pow maximum minimum` and many more) use NumPy-style broadcasting. A reduction with no axis reduces everything to a 0-d tensor (`tensor[]`). Use `nn.item(t)` to get a plain number from a one-element tensor.

Broadcasting shapes follow the NumPy rule, and `bmatmul` broadcasts batch dimensions:

```
print(nn.shape(nn.add(nn.ones([4, 1, 3]), nn.ones([5, 1]))));          // [4,
5, 3]
print(nn.shape(nn.bmatmul(nn.ones([2, 1, 3, 4]), nn.ones([5, 4, 2])))); //
[2, 5, 3, 2]
```

2.2 Function families

Family	Functions
Creation	`tensor zeros ones full rand randn arange linspace eye`

Family	Functions
Shaping	`reshape transpose concat stack split squeeze unsqueeze swapaxes moveaxis broadcast_to slice slice_axis flip roll tile repeat pad take gather scatter scatter_add tril triu diag`
Reductions	`sum mean max min var std prod argmax argmin any all logsumexp median quantile norm`
Scans and sorting	`cumsum cumprod cummax cummin diff sort argsort topk` (sorting is stable, NaN sorts last)
Special functions	`erf erfinv lgamma digamma normal_cdf normal_ppf logit log_sigmoid log1p expm1`
Products	`matmul bmatmul tensordot einsum outer kron cross`
Statistics	`cov corrcoef cdist cosine_similarity normalize histogram bincount convolve`
Sampling	`uniform normal trunc_normal bernoulli randperm multinomial`

A few results to calibrate your intuition:

```
let t = nn.tensor([3, 1, 4, 1, 5, 9, 2, 6]);
print(nn.sort(t));       // [1, 1, 2, 3, 4, 5, 6, 9]
print(nn.argsort(t));    // [1, 3, 6, 0, 2, 4, 7, 5]   (stable: the two 1s
keep their order)
print(nn.cumsum(t));     // [3, 4, 8, 9, 14, 23, 25, 31]
print(nn.median(t));     // 3.5
print(nn.topk(nn.tensor([3, 9, 1, 7]), 2).values);   // [9, 7]
print(nn.logsumexp(nn.tensor([1.0, 2.0, 3.0])));     // 3.40761
```

2.3 einsum

`nn.einsum(spec, t...)` takes up to 8 operands. Two-operand contractions run on GEMM, and everything else uses a generic loop nest. There is no `...` ellipsis. Scaled dot-product attention scores for a batch, with no loops:

```
let Q  = nn.randn([2, 3, 4]);
let Kt = nn.randn([2, 5, 4]);
let w  = nn.softmax(nn.einsum("bqd,bkd->bqk", Q, Kt), -1);
print(nn.shape(w), nn.allclose(nn.sum(w, -1), nn.ones([2, 3])));   // [2, 3,
5] true
```

Note that `nn.softmax(t, axis)` is the tensor function, while `nn.softmax()` with **no arguments** creates the softmax *layer*.

2.4 Linear algebra

Square-matrix routines accept batches (`[..., n, n]`). Decompositions that return several things take one 2-D matrix and return a struct.

Function	Returns
`det inv solve slogdet`	LU with partial pivoting. A singular matrix is an error.
`lu cholesky chol_solve qr`	factorizations (`{p,l,u}`, lower factor, `{q,r}`)
`eigh eigvalsh`	symmetric eigendecomposition: `{values (ascending), vectors (columns)}`
`svd svdvals pinv lstsq`	SVD `{u, s, vt}`, pseudo-inverse, minimum-norm least squares `{x, rank, s}`
`matrix_rank cond matrix_norm matrix_power expm`	scalar summaries and matrix functions

```
let M = nn.tensor([[4.0, 1.0], [2.0, 3.0]]);
print(nn.det(M));                              // 10
print(nn.inv(M));                              // [[0.3, -0.1], [-0.2, 0.4]]
print(nn.solve(M, nn.tensor([1.0, 2.0])));     // [0.1, 0.6]
print(nn.svd(nn.tensor([[3.0, 0.0], [0.0, 4.0]])).s);   // [4, 3]
(descending)
print(nn.matrix_rank(nn.tensor([[1.0, 2.0], [2.0, 4.0]])));  // 1
```

PCA in four lines. `cov` takes `[observations, variables]`:

```
let pts = nn.tensor([[2.5,2.4],[0.5,0.7],[2.2,2.9],[1.9,2.2],[3.1,3.0],
                     [2.3,2.7],[2.0,1.6],[1.0,1.1],[1.5,1.6],[1.1,0.9]]);
let ev  = nn.eigh(nn.cov(pts));
print(ev.values);                        // [0.0490834, 1.28403]  (ascending)
let top = nn.slice_axis(ev.vectors, 1, 1, 2);   // the leading direction
(last column)
let low = nn.matmul(pts, top);                  // 10 x 1 projection
```

The Jacobi eigen and SVD solvers are accurate and always converge, but cost O(n^3) per sweep: fine to a few hundred rows or columns, not a LAPACK replacement for thousands. There are no complex numbers, FFT or general non-symmetric eigenproblems.

2.5 Data preparation

`one_hot(labels, classes)`, `standardize`, `minmax`, `shuffle`, `randint`, `train_test_split(x, y, frac)` (returns `[x_tr, y_tr, x_te, y_te]`), `pad_sequences`, and `window_series(data, window [, horizon])`, which returns `[X[S, window, F], Y[S, horizon*F]]` for forecasting:

```
let w = nn.window_series([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 3);
print(nn.shape(w[0]), nn.shape(w[1]));    // [7, 3, 1] [7, 1]
print(nn.one_hot([0, 2, 1], 3));          // [[1,0,0],[0,0,1],[0,1,0]]
```

---

3. Layers and models

3.1 The layer catalogue

Group	Constructors	Shape contract
Dense	`dense(in, out [, "bias", "init"])`	`[..., in]` to `[..., out]`: applies to the last axis, so it is time-distributed on sequences
Activations	`relu relu6 leaky_relu elu selu gelu sigmoid tanh softplus silu mish softsign hard_sigmoid softmax log_softmax`, `activation(name [, p])`	same shape; softmax acts on the last axis
Regularization	`dropout(p)`, `batchnorm(features)`, `layernorm(features)`	`batchnorm` is per channel over `[N, C, ...]`; its running statistics are saved with the model

Group	Constructors	Shape contract
Reshaping	`flatten()`, `reshape_layer(shape)`	per-sample shapes, batch axis kept
Embedding	`embedding(vocab, dim [, "padding_idx"])`	ids `[N]` or `[N, T]` to `[N, (T,) dim]`; the padding row gets no gradient
Convolution	`conv1d`, `conv2d(in, out, k [, "stride", "padding", "dilation"])`	`[N, C, L]` and `[N, C, H, W]`; padding may be `"same"`
Pooling	`maxpool1d maxpool2d avgpool1d avgpool2d global_avg_pool`	`global_avg_pool` gives `[N, C]`
Recurrent	`rnn lstm gru (in, hidden [, "return_sequences"])`	`[N, T, F]` to `[N, H]` or `[N, T, H]`; full BPTT, orthogonal recurrent weights, LSTM forget bias 1
Attention	`attention(d_model, heads [, "causal"])`, `pos_encoding()`	`[N, T, D]` to `[N, T, D]`; full O(T^2) attention

Initializers (option `"init"`): `xavier` (default), `xavier_normal`, `he` / `kaiming`, `he_uniform`, `lecun`, `lecun_uniform`, `normal`, `uniform`, `zeros`, `ones`, `constant`, `orthogonal`.

3.2 Containers and builders

- `sequential([layers])` is the model type. `append(model, layer)` and `layers(model)` edit and inspect it.
- `residual([layers])` computes `y = x + f(x)`. The shapes must match.
- `mlp(sizes [, "activation", "out_activation", "dropout", "batchnorm", "layernorm", "init"])` picks an initializer that matches the activation.
- `transformer_block(d_model, heads, ff_dim [, "dropout", "causal"])` is a pre-norm block: `x + Attn(LN(x))`, then `x + FFN(LN(x))` with GELU.
- A layer can belong to **one model only**. Use `nn.clone` to duplicate a whole model.

3.3 Reading a model with summary

`nn.summary(model, input_shape)` **returns a string** (print it). The shape is per sample, without the batch axis:

```
nn.seed(3);
let cnn = nn.sequential([
    nn.conv2d(1, 4, 3, "padding", 1), nn.batchnorm(4), nn.relu(),
nn.maxpool2d(2),
    nn.conv2d(4, 8, 3, "padding", 1), nn.relu(), nn.global_avg_pool(),
nn.dense(8, 2)]);
print(nn.summary(cnn, [1, 8, 8]));
```

```
Model: sequential (8 layers)
  #  Layer                              Output shape           Params
  0  conv2d 1->4 k=3x3 s=1 p=1          [4,8,8]                    40
  1  batchnorm 4                        [4,8,8]                     8
  2  relu                               [4,8,8]                     0
  3  maxpool2d k=2 s=2                  [4,4,4]                     0
  4  conv2d 4->8 k=3x3 s=1 p=1          [8,4,4]                   296
  5  relu                               [8,4,4]                     0
  6  global_avg_pool                    [8]                         0
  7  dense 8->2                         [2]                        18
Total params: 362  (trainable: 362, frozen: 0, buffers: 8)
```

Batch normalization's running mean and variance are **buffers** (8 here), not trainable parameters. Use these formulas to check a summary by hand:

Layer	Parameters
`dense(i, o)`	`i*o + o`
`conv2d(ci, co, k)`	`k*k*ci*co + co`
`batchnorm(c)`, `layernorm(c)`	`2c`
`embedding(v, d)`	`v*d`
`lstm(i, h)`	`4h(i + h + 1)`
`gru(i, h)`	`3h(i + h + 1)`
`attention(d, heads)`	`4d^2 + 4d` (query, key, value and output projections with biases)

For example `nn.count_params(nn.mlp([784, 256, 128, 10]))` is `235146`, and `lstm(1, 16)` followed by `dense(16, 1)` has `1169`.

---

4. Losses, optimizers and schedules

4.1 Losses

Use `nn.loss(name [, options])`, or pass the name as a string wherever a loss is expected. All losses reduce by the mean.

Name	Prediction and target	Options
`mse mae logcosh`	same shape	
`huber`	same shape	`"delta"` (default 1)
`bce`	probabilities and 0..1 targets	
`bce_logits`	logits and 0..1 targets, numerically stable	`"pos_weight"`
`ce`	logits `[.., C]` and class indices **or** a distribution	`"smoothing"`, `"ignore_index"`; works for sequences `[N, T, C]` against `[N, T]`
`nll`	log-probabilities and class indices	`"ignore_index"`
`kl`	log-probabilities and probabilities	
`hinge`	scores and +1/-1	

`nn.loss_value(loss, pred, target)` returns a plain float, which is handy for hand checks:

```
print(nn.loss_value("ce", [[2.0, 1.0, 0.1]], [0]));          // 0.41703
print(nn.loss_value("mse", [[1.0, 2.0]], [[0.0, 0.0]]));     // 2.5
print(nn.loss_value("mae", [[1.0, 2.0]], [[0.0, 0.0]]));     // 1.5
print(nn.loss_value("huber", [[3.0]], [[0.0]]));             // 2.5   (delta
1: 1 * (3 - 0.5))
```

4.2 Optimizers

`sgd adam adamw rmsprop adagrad (model [, lr] [, options])`.

Option	Applies to	Default
`"momentum"`, `"nesterov"`	sgd, rmsprop	0, false
`"weight_decay"` (`"wd"`)	all (decoupled for adamw)	0 (adamw 0.01)
`"beta1"`, `"beta2"`, `"eps"`, `"amsgrad"`	adam, adamw	0.9, 0.999, 1e-8, false
`"alpha"`, `"centered"`	rmsprop	0.99, false
`"decay_all"`	all	false: biases and norm parameters are **not** decayed by default

`set_lr`, `get_lr`, `clip_grad_norm(model, max)` (returns the norm before clipping) and `clip_grad_value`.

4.3 Learning-rate schedules

The `"schedule"` option of `fit`, or `nn.lr_at(spec, base, epoch, total [, warmup])`: `const`, `step:E:G`, `exp:G`, `cosine[:min]`, `linear[:end]`, `poly:p[:min]`, and `plateau:patience:factor[:min]`. `"warmup"` adds a linear ramp. `lr_at` lets you see a schedule without training:

```
print(nn.lr_at("cosine", 0.1, 0, 10));        // 0.1
print(nn.lr_at("cosine", 0.1, 5, 10));        // 0.05
print(nn.lr_at("cosine", 0.1, 10, 10));       // 0
print(nn.lr_at("linear", 0.1, 5, 10));        // 0.05
print(nn.lr_at("exp:0.9", 1.0, 2, 10));       // 0.81
print(nn.lr_at("step:3:0.5", 0.1, 7, 10));    // 0.025   (halved twice)
```

---

5. Training

5.1 fit: the batteries-included loop

`nn.fit(model, loss, optimizer, X, Y, options...)` returns a **History**. The loss and optimizer may be handles or names; with a name, optimizer options such as `"lr"` go in the same list.

Option	Meaning	Default
`"epochs"`, `"batch_size"`		10, 32

Option	Meaning	Default
`"shuffle"`, `"drop_last"`		true, false
`"val_split"` or `"val", [Xv, Yv]`	held-out validation	none
`"metric"`	`accuracy`, `top_k:N`, `mae`, `mse`, `rmse`, `r2`, `none`	accuracy for ce/nll/bce/hinge
`"schedule"`, `"warmup"`	LR schedule	const, 0
`"clip_norm"`, `"clip_value"`	gradient clipping	off
`"accum"`	gradient accumulation steps	1
`"patience"`, `"min_delta"`, `"restore_best"`	early stopping; restores the best weights when `patience` is set	off
`"callback"`	`fn(epoch, loss, val_loss, metric) -> bool`; return `true` to stop	none
`"verbose"`	0 is silent, n prints every n epochs	1

The History has `epochs`, `loss`, `val_loss`, `metric`, `val_metric`, `lr` (arrays), `best_epoch` (1-based), `best_loss`, `stopped_early`, `seconds` and `metric_name`. A NaN or infinite loss stops training with an error that names the epoch and suggests a fix.

5.2 Example: XOR

```
nn.seed(42);
let X = [[0, 0], [0, 1], [1, 0], [1, 1]];
let Y = [[0], [1], [1], [0]];
let net = nn.mlp([2, 8, 1], "activation", "tanh", "out_activation",
"sigmoid");
let h = nn.fit(net, "bce", "adam", X, Y, "epochs", 400, "batch_size", 4,
"lr", 0.05, "verbose", 0);
print(len(h.loss), h.loss[399] < 0.05, h.metric_name);   // 400 true acc
print(nn.predict_class(net, X));                         // [0, 1, 1, 0]
```

`predict_class` takes the argmax over the last axis, and thresholds at 0.5 (configurable) when the output has one unit, as here. The summary of this network reports `Total params: 33`.

5.3 Example: regression with everything switched on

```
nn.seed(5);
let xs = []; let ys = [];
for (i in 0..200) { let x = -3.0 + 6.0 * i / 199; xs.push([x]);
ys.push([sin(x)]); }

let net = nn.mlp([1, 32, 32, 1], "activation", "tanh");
let h = nn.fit(net, "mse", "adam", xs, ys,
    "epochs", 150, "batch_size", 32, "lr", 0.01,
    "schedule", "cosine:0.0001", "warmup", 5,
    "val_split", 0.2, "patience", 15, "clip_norm", 1.0,
    "metric", "r2", "verbose", 0,
    "callback", fn(ep, loss, vloss, met) -> bool { return false; });
print(h.metric_name, h.val_metric[len(h.val_metric) - 1] > 0.9);   // r2 true
```

The callback sees every epoch and can stop training by returning `true`. Here it declines, so the early-stopping `patience` decides. `h.lr` holds the learning rate used in every epoch, so you can plot the warmup and cosine decay.

5.4 Manual training loops

When you need custom batches, targets or losses (reinforcement learning, for example), drive the pieces yourself:

```
let m    = nn.mlp([1, 4, 1], "activation", "tanh");
let opt  = nn.adam(m, 0.05);
let loss = nn.loss("mse");
var last = 0.0;
for (i in 0..200) {
    let out = nn.forward(m, [[0.5], [1.0]]);              // training mode by
default
    last = nn.loss_value(loss, out, [[1.0], [2.0]]);
    nn.zero_grad(m);
    nn.backward(m, loss);
    nn.step(opt);
    nn.free(out);                                         // required in
manual loops
}
print(last < 0.01);   // true
```

The one-call form `nn.train_step(model, loss, opt, x, y)` does forward, loss, backward and step and returns the loss. Use `nn.clip_grad_norm(model, max)` between `backward` and `step` for clipping in a manual loop.

5.5 Verifying a model with gradcheck

`nn.gradcheck(model, loss, x, y [, "eps", "samples"])` compares analytic and numerical gradients and returns `{ok, max_rel_error, checked, worst}`. Run it whenever you build an unusual architecture:

```
let g = nn.gradcheck(net, "bce", X, Y);
print(g.ok, g.max_rel_error < 1e-4, g.checked);   // true true 41
```

The XOR network has 33 parameters, and `gradcheck` sampled 41 checks. If `ok` is false, `g.worst` names the parameter with the largest relative error, which tells you which layer has a bug or which input is numerically unstable.

---

6. Architectures

6.1 Convolutional classifier

The summary in section 3.3 is a complete small CNN. Training it is one call, because `fit` accepts image tensors shaped `[N, C, H, W]`:

```
let cnn = nn.sequential([
    nn.conv2d(1, 16, 3, "padding", 1), nn.batchnorm(16), nn.relu(),
nn.maxpool2d(2),
    nn.conv2d(16, 32, 3, "padding", 1), nn.relu(), nn.global_avg_pool(),
nn.dense(32, 10)]);
nn.fit(cnn, "ce", "adam", images, labels, "epochs", 20, "val_split", 0.1,
"patience", 4);
```

Shape arithmetic is easy to check with a zero tensor in inference mode:

```
let c = nn.sequential([nn.conv2d(3, 8, 3, "stride", 2, "padding", 1)]);
print(nn.shape(nn.forward(c, nn.zeros([2, 3, 32, 32]), false)));      // [2,
8, 16, 16]
print(nn.shape(nn.forward(nn.sequential([nn.maxpool2d(2)]), nn.zeros([1, 1,
8, 8]), false)));  // [1, 1, 4, 4]
```

Batch normalization uses batch statistics inside `fit` and the running statistics in `predict` and `evaluate`. If you write a manual loop, pass `false` as the third argument of `forward` for evaluation, otherwise you will normalize with the statistics of the batch you are evaluating.

6.2 Recurrent models and forecasting

`rnn`, `lstm` and `gru` take `[N, T, F]`. Without `"return_sequences"` they return the last hidden state `[N, H]`. With it they return every step `[N, T, H]`:

```
let seq  = nn.sequential([nn.lstm(4, 6, "return_sequences", true)]);
print(nn.shape(nn.forward(seq,  nn.zeros([2, 5, 4]), false)));        // [2,
5, 6]
let last = nn.sequential([nn.lstm(4, 6)]);
print(nn.shape(nn.forward(last, nn.zeros([2, 5, 4]), false)));        // [2,
6]
```

A forecasting pipeline is three lines with `window_series`. Clip gradients, since recurrent models are prone to exploding gradients:

```
let w = nn.window_series(series, 24);                    // 24-step windows -
> next value
let m = nn.sequential([nn.lstm(1, 32), nn.dense(32, 1)]);
nn.fit(m, "mse", "adam", w[0], w[1], "epochs", 30, "clip_norm", 1.0);
```

For multi-step prediction pass a `horizon` to `window_series`, or feed predictions back in recursively.

6.3 Transformers

An autoregressive language model is an embedding, positional codes, a stack of causal blocks, a final norm and a projection to the vocabulary:

```
let lm = nn.sequential([
    nn.embedding(20, 16), nn.pos_encoding(),
    nn.transformer_block(16, 2, 32, "causal", true),
    nn.layernorm(16), nn.dense(16, 20)]);
print(nn.summary(lm, [6]));
let y = nn.forward(lm, nn.randint(0, 20, [2, 6]), false);
print(nn.shape(y));   // [2, 6, 20]
```

```
Model: sequential (5 layers)
  #  Layer                                Output shape           Params
  0  embedding 20 x 16                    [6,16]                    320
  1  pos_encoding                         [6,16]                      0
  2  sequential                           [6,16]                   2224
  3    residual                           [6,16]                   1120
  4      sequential                       [6,16]                   1120
  5        layernorm 16                   [6,16]                     32
  6        attention d=16 heads=2 causal  [6,16]                   1088
  7    residual                           [6,16]                   1104
  8      sequential                       [6,16]                   1104
  9        layernorm 16                   [6,16]                     32
 10        dense 16->32                   [6,32]                    544
 11        gelu                           [6,32]                      0
 12        dense 32->16                   [6,16]                    528
 13  layernorm 16                         [6,16]                     32
 14  dense 16->20                         [6,20]                    340
Total params: 2916  (trainable: 2916, frozen: 0, buffers: 0)
```

Train it with `"ce"` on shifted targets: the loss accepts logits `[N, T, C]` against class indices `[N, T]`. Generate text with `nn.sample(logits, temperature, top_k)`, which draws a token id from `softmax(logits / temperature)` restricted to the `top_k` largest logits. Attention is full O(T^2); there is no cross-attention or bidirectional RNN wrapper.

6.4 Text classification

The standard pipeline with `std.nlp`: tokenize, map to ids, `pad_sequences`, then `embedding`, `gru`, `dense`, and `fit(..., "ce", ...)`. Give the embedding a `"padding_idx"` so padded positions receive no gradient.

---

7. Model management

7.1 Saving and loading

Function	What it stores
`save(model, path)` / `load(path)`	Architecture, weights, batch-norm buffers and frozen flags
`save_weights(model, path)` / `load_weights(model, path)`	Weights only, into an identical architecture; all or nothing

```
nn.save(net, "xor.kcnn");
let net2 = nn.load("xor.kcnn");
print(nn.predict_class(net2, X));                                    //
[0, 1, 1, 0]
print(nn.allclose(nn.forward(net, X, false), nn.forward(net2, X, false))); //
true
```

The file format is `KCNN` magic, a format version, a layer tree, and a CRC-32 trailer. It is little-endian with 64-bit floats, so a reloaded model reproduces predictions **bit for bit**. Writes are atomic (temp file plus rename). Loading never trusts a size field: truncated, bit-flipped or hand-edited files give clear errors such as `file is truncated` or `checksum mismatch`. **Optimizer state is not saved**, so resuming training restarts Adam's moments.

7.2 Inspecting and copying parameters

```
print(nn.param_names(net));   // [0.dense.weight, 0.dense.bias,
2.dense.weight, 2.dense.bias]
print(nn.count_params(net));  // 33
```

`get_params(model)` returns the parameter tensors, `set_params(model, list)` installs them, and `clone(model)` makes an independent deep copy. `nn.copy_weights(dst, src [, tau])` does a hard copy, or a soft (Polyak) update `dst = tau * src + (1 - tau) * dst`:

```
let a = nn.sequential([nn.dense(1, 1, "init", "zeros")]);
let b = nn.sequential([nn.dense(1, 1, "init", "ones")]);
nn.copy_weights(a, b, 0.25);
print(nn.get_params(a));      // [tensor[1,1] [[0.25]], tensor[1] [0]]
nn.copy_weights(a, b);        // no tau: a hard copy
print(nn.get_params(a));      // [tensor[1,1] [[1]], tensor[1] [0]]
```

Note that bias parameters are zero in both models here, so the soft update leaves the bias at 0.

7.3 Transfer learning

`nn.freeze(model [, layer_index])` and `nn.unfreeze` mark layers as frozen. Every optimizer skips frozen layers, and the frozen flags are saved in the model file:

```
nn.freeze(lm, 0);                     // freeze the embedding
print(nn.count_params(lm, true));     // 2596  (trainable only)
print(nn.count_params(lm));           // 2916
```

The usual recipe is to pretrain an autoencoder or language model, freeze the encoder, attach a new `dense` head and `fit` on the small labeled set.

---

8. Reinforcement learning: DQN and Double DQN

`std.nn` has no RL environment API, but it has everything a value-based agent needs: `clone` and `copy_weights` for target networks, `forward` for batched Q-values, `argmax` for greedy actions, and `train_step` for the regression step. You write the environment and the replay buffer in plain KingCobra.

8.1 The algorithms

A Q-network maps a state to one value per action. DQN regresses the Q-value of the taken action toward a bootstrapped target. With reward `r`, discount `gamma`, a done flag `d`, online parameters `theta` and target parameters `theta_minus`:

```
y_{\mathrm{DQN}} = r + \gamma\,(1 - d)\,\max_{a'} Q_{\theta^-}(s', a')
```

The `max` over the **same** noisy estimates both selects and evaluates the action, which makes the target biased upward (maximization bias). **Double DQN** decouples the two roles: the online network *selects* the action and the target network *evaluates* it:

```
y_{\mathrm{DDQN}} = r + \gamma\,(1 - d)\,Q_{\theta^-}\!\left(s',\
\arg\max_{a'} Q_{\theta}(s', a')\right)
```

The change is one line of code, but it needs **three** forward passes per minibatch instead of two: `Q_online(s)`, `Q_online(s')` for the argmax, and `Q_target(s')` for the value.

8.2 Worked numbers

Take `r = 0.5`, `gamma = 0.9`, `d = 0`, with `Q_online(s') = [1, 3, 2]` and `Q_target(s') = [5, 2, 4]`.

Method	Chosen action	Target
DQN	`argmax Q_target` = action 0	`0.5 + 0.9 * 5 = 5.0`
Double DQN	`argmax Q_online` = action 1	`0.5 + 0.9 * 2 = 2.3`

The target network's own favorite action (5) is overestimated, while the online network does not agree that action 0 is best, so Double DQN returns the more conservative 2.3.

8.3 Full program: Double DQN on a grid world

The world is 4 by 4. The agent starts at state 0 (top left). State 5 is a pit (reward -1, episode ends) and state 15 is the goal (+1, episode ends). Every other step costs -0.04. Actions are 0 up, 1 down, 2 left, 3 right, and walls just block movement. The state is a one-hot vector of length 16.

```
import std.nn;

#define CAP 1000
#define BATCH 32
#define STATES 16
#define ACTIONS 4

fn rnd() -> float {
    let t = nn.rand([1]);
    let v = nn.item(t);
    nn.free(t);                       // every tensor handle must be released
    return v;
}

fn onehot(s: int) -> array {
    var v = [0.0; STATES];
    v[s] = 1.0;
    return v;
}

// returns [next_state, reward, done]
fn step(s: int, a: int) -> array {
    var r = idiv(s, 4);
    var c = s % 4;
    if (a == 0 && r > 0) { r--; }
    if (a == 1 && r < 3) { r++; }
    if (a == 2 && c > 0) { c--; }
    if (a == 3 && c < 3) { c++; }
    let ns = r * 4 + c;
    if (ns == 15) { return [ns, 1.0, 1]; }
    if (ns == 5)  { return [ns, -1.0, 1]; }
    return [ns, -0.04, 0];
}

// circular replay buffer: parallel arrays inside one struct
struct Replay { s: array, a: array, r: array, n: array, d: array, head: int }

impl Replay {
    fn new() -> Replay { return Replay { s: [], a: [], r: [], n: [], d: [],
```

```
head: 0 }; }
    fn size(self) -> int { return len(self.s); }
    fn add(self, s: int, a: int, r: float, n: int, d: int) {
        if (len(self.s) < CAP) {
            self.s.push(onehot(s)); self.a.push(a); self.r.push(r);
            self.n.push(onehot(n)); self.d.push(d);
        } else {
            let h = self.head;
            self.s[h] = onehot(s); self.a[h] = a; self.r[h] = r;
            self.n[h] = onehot(n); self.d[h] = d;
            self.head = (h + 1) % CAP;
        }
    }
}

fn main() {
    nn.seed(8);
    let online = nn.mlp([STATES, 32, ACTIONS], "activation", "relu");
    let target = nn.clone(online);          // independent copy that trails
the online net
    let opt    = nn.adam(online, 0.01);
    let buf    = Replay.new();
    let gamma = 0.95;
    let tau = 0.05;
    let episodes = 600;
    var wins = 0;

    for (ep in 0..episodes) {
        let eps = 1.0 - 0.95 * ep / episodes;     // epsilon decays from 100%
to 5%
        var s = 0;
        var done = 0;
        var t = 0;
        while (done == 0 && t < 30) {
            var a = 0;
            if (rnd() < eps) { a = min(int(rnd() * ACTIONS), ACTIONS - 1); }
            else { a = nn.predict_class(online, [onehot(s)])[0]; }
            let res = step(s, a);
            buf.add(s, a, res[1], res[0], res[2]);

            if (buf.size() >= 64) {
                // 1. sample a minibatch
                let idx = [];
                for (v in nn.to_array(nn.randint(0, buf.size(), [BATCH]))) {
idx.push(int(v)); }
                let S = []; let S2 = [];
```

```
                for (k in idx) { S.push(buf.s[k]); S2.push(buf.n[k]); }

                // 2. three forward passes, all in inference mode
                let q   = nn.forward(online, S,  false);   // Q_online(s, .)
                let qo2 = nn.forward(online, S2, false);   // Q_online(s', .)
selects the action
                let qt2 = nn.forward(target, S2, false);   // Q_target(s', .)
evaluates it

                // 3. build Double DQN targets: only the taken action gets a
new value
                let Y     = nn.to_array(q);
                let best  = nn.to_array(nn.argmax(qo2, 1));
                let qeval = nn.to_array(qt2);
                for (i in 0 .. BATCH) {
                    let k = idx[i];
                    let nxt = qeval[i][int(best[i])];
                    Y[i][buf.a[k]] = buf.r[k] + gamma * (1 - buf.d[k]) * nxt;
                }
                nn.free(q); nn.free(qo2); nn.free(qt2);

                // 4. regress, then soft-update the target
                nn.train_step(online, "huber", opt, S, Y);
                nn.copy_weights(target, online, tau);
            }
            if (res[2] == 1) { done = 1; if (res[1] > 0) { wins++; } }
            s = res[0];
            t++;
        }
    }

    // print the greedy policy and follow it from the start
    let all = [];
    for (st in 0 .. STATES) { all.push(onehot(st)); }
    let pol = nn.predict_class(online, all);
    let arrows = ["^", "v", "<", ">"];
    for (r in 0..4) {
        var line = "";
        for (c in 0..4) {
            let k = r * 4 + c;
            if (k == 15) { line = line + "G "; }
            else if (k == 5) { line = line + "X "; }
            else { line = line + arrows[pol[k]] + " "; }
        }
        print(line);
    }
```

```
    var s = 0; var total = 0.0; var steps = 0; var fin = 0;
    while (fin == 0 && steps < 20) {
        let res = step(s, pol[s]);
        s = res[0]; total += res[1]; steps++;
        if (res[2] == 1) { fin = 1; }
    }
    print("wins:", wins, "greedy return:", total, "steps:", steps);
}
```

Output (the run is deterministic because of `nn.seed(8)`):

```
v > > v
v X v v
> > > v
> > > G
wins: 360 greedy return: 0.8 steps: 6
```

The best possible route avoids the pit and takes 6 steps: five steps cost -0.04 each and the last reaches the goal for +1, so the optimal return is 0.8, and the agent found it. The policy from states that the optimal route never visits (for example the arrow at state 3) does not matter.

8.4 Design notes for experts

- **Memory in the loop.** Each `forward` result is freed right after `to_array`. The three `nn.free` calls are essential. `predict_class` and `train_step` manage their own temporaries.
- **Target updates.** The soft update `tau = 0.05` every step is smoother than copying all weights every N steps. For a hard update, call `nn.copy_weights(target, online)` with no `tau` every N steps.
- **Loss.** `"huber"` limits the gradient from large TD errors, which stabilizes training compared with `"mse"`. The targets only differ from the network output at the taken action, so the other three outputs get zero error, exactly as in the reference algorithm.
- **The macro gotcha.** A range whose end is a macro, such as `0..BATCH` fails with an undefined-variable error. Write `0 .. BATCH` with spaces when the range end is a macro. `0..4` with a literal is fine.
- **Exploration.** Epsilon decays linearly here. Heavier tasks usually use an exponential decay with a floor.
- **Larger problems.** Replace the one-hot input with image tensors and a `conv2d` stack, use `"clip_norm"`-style clipping through `nn.clip_grad_norm(online, 10.0)` between `backward` and `step` if you use the manual loop, and add prioritized replay by keeping a priority array next to the buffers.

8.5 Variants to try

- Vanilla DQN: replace `best[i]` by `argmax` of `qt2` (use `nn.max(qt2, 1)`). The project's `examples/nn/07_dqn_gridworld.kc` does this and reaches the same return of 0.8 on this tiny world, because overestimation hardly matters when rewards are deterministic. The difference shows up with noisy rewards or many actions.
- Dueling heads: build two `sequential` branches and combine value and advantage with tensor math.
- N-step returns: store the discounted sum of the next N rewards and use `gamma^N` in the target.

---

9. Performance and pitfalls

9.1 What to expect from one core

`std.nn` is single-threaded CPU code in float64. The project documentation reports measured training throughput on one core of about 6,000 samples per second for a 784-256-128-10 MLP, 3,500 for a small 28 by 28 CNN with 8 and 16 filters, 1,300 for an LSTM (16 inputs, 64 units, 30 steps) and 1,000 for a d=64 transformer block at T=32. Treat these as orders of magnitude: your hardware will differ. Models of this size train in seconds to minutes. Large image or language models are out of reach, by design.

9.2 Common mistakes

Mistake	Symptom	Fix
Evaluating with `nn.forward(m, x)`	Dropout is active, batch norm uses batch statistics, results are noisy	Pass `false` as the third argument, or use `predict` or `evaluate`
Not freeing per-step tensors in a manual loop	Memory grows every step	`nn.free(out)` after reading each `forward` output
Using the core `free()` on a tensor	Error: cannot free a value of type handle	Use `nn.free`
Calling `nn.softmax()` when you want the function	You get a layer handle	Use `nn.softmax(t, axis)` for tensors
Printing `nn.summary(...)` without `print`	Nothing appears, because it returns a string	`print(nn.summary(m, shape))`

Mistake	Symptom	Fix
Putting one layer into two models	Error: a layer can belong to one model	Build separate layers or use `nn.clone(model)`
Passing a per-batch shape to `summary`	Wrong shapes	Give the per-sample shape without the batch axis
Typo in an option name	Error listing the valid options	Read the list in the message
Learning rate too high	The run stops with an error naming the epoch of the NaN or infinite loss	Lower `lr`, add `"clip_norm"`, add warmup
Reading the confusion matrix the wrong way round	Precision and recall swapped	Rows are the true class, columns the predicted class
Expecting a saved model to resume Adam exactly	Training jolts after reload	Optimizer state is not stored

9.3 A verification checklist for a new architecture

1. Print `nn.summary(model, shape)` and check the output shape of every layer against the shape you expect.
2. Verify the parameter count by hand with the formulas in section 3.3.
3. Run `nn.gradcheck` on a tiny batch. If it fails, `g.worst` names the culprit.
4. Overfit eight samples to near-zero loss. If you cannot, the model or loss is wrong.
5. Only then scale up with `val_split`, a schedule and early stopping.

---

10. Work problems

Each problem tells you what to compute and the answer shown is **only the final result**. No solutions are given: write the code (or work it out by hand), run it, and compare. Problems that depend on random initialization are stated with their seed, and shapes and parameter counts are exact on every run.

10.1 Parameter counts

Problem	Answer
`nn.count_params(nn.mlp([4, 8, 3]))`	67

Problem	Answer
`nn.count_params(nn.mlp([784, 256, 128, 10]))`	235146
Parameters of `nn.conv2d(1, 4, 3)`	40
Parameters of `nn.conv2d(4, 8, 3)`	296
Parameters of `nn.batchnorm(4)` (trainable)	8
`lstm(1, 16)` followed by `dense(16, 1)`	1169
Parameters of `nn.gru(3, 5)`	135
Parameters of `nn.attention(16, 2)`	1088
Parameters of `nn.embedding(20, 16)`	320
Parameters of `nn.layernorm(16)`	32
The language model of section 6.3: total parameters	2916
The same model after `nn.freeze(lm, 0)`: trainable parameters	2596
The XOR network `mlp([2, 8, 1])`: total parameters	33

10.2 Shapes

Problem	Answer
`conv2d(3, 8, 3, "stride", 2, "padding", 1)` on `[2, 3, 32, 32]`	[2, 8, 16, 16]
`maxpool2d(2)` on `[1, 1, 8, 8]`	[1, 1, 4, 4]
`lstm(4, 6)` on `[2, 5, 4]`	[2, 6]
`lstm(4, 6, "return_sequences", true)` on `[2, 5, 4]`	[2, 5, 6]
The language model of section 6.3 on token ids `[2, 6]`	[2, 6, 20]
`window_series` of the numbers 1 to 10 with window 3: shapes of X and Y	[7, 3, 1] and [7, 1]

Problem	Answer
`nn.add` of shapes `[4, 1, 3]` and `[5, 1]`	[4, 5, 3]
`nn.bmatmul` of `[2, 1, 3, 4]` and `[5, 4, 2]`	[2, 5, 3, 2]
`nn.concat` of `[2, 3]` and `[4, 3]` along axis 0	[6, 3]
`nn.stack` of two `[2, 3]` tensors along axis 0	[2, 2, 3]
`nn.einsum("bqd,bkd->bqk", ...)` of `[2, 3, 4]` and `[2, 5, 4]`	[2, 3, 5]

10.3 Tensor math

Let `a = [[1, 2, 3], [4, 5, 6]]`.

Problem	Answer
`a` times its transpose (`matmul`)	[[14, 32], [32, 77]]
`nn.sum(a, 0)`	[5, 7, 9]
`nn.mean(a, 1)`	[2, 5]
`nn.argmax(a, 1)`	[2, 2]
`nn.softmax` of `[1, 2, 3]`	[0.0900306, 0.244728, 0.665241]
`nn.logsumexp` of `[1, 2, 3]`	3.40761
`nn.det` of `[[4, 1], [2, 3]]`	10
`nn.inv` of `[[4, 1], [2, 3]]`	[[0.3, -0.1], [-0.2, 0.4]]
`nn.solve` of that matrix with right-hand side `[1, 2]`	[0.1, 0.6]
`nn.norm` of `[3, 4]`	5
Singular values of `diag(3, 4)` (descending)	[4, 3]
`nn.matrix_rank` of `[[1, 2], [2, 4]]`	1
`nn.one_hot([0, 2, 1], 3)`	[[1, 0, 0], [0, 0, 1], [0, 1, 0]]

Let `t = [3, 1, 4, 1, 5, 9, 2, 6]`.

Problem	Answer
`nn.sort(t)`	[1, 1, 2, 3, 4, 5, 6, 9]
`nn.argsort(t)`	[1, 3, 6, 0, 2, 4, 7, 5]
`nn.cumsum(t)`	[3, 4, 8, 9, 14, 23, 25, 31]
`nn.median(t)`	3.5
`nn.where(nn.gt(t, 3), t, 0)`	[0, 0, 4, 0, 5, 9, 0, 6]
`nn.topk(t, 2).values`	[9, 6]

PCA: the eigenvalues of `nn.cov(pts)` for the ten points in section 2.4 are **[0.0490834, 1.28403]**.

10.4 Losses, schedules and metrics

Problem	Answer
`ce` loss of logits `[[2, 1, 0.1]]` with target class 0	0.41703
`mse` of prediction `[[1, 2]]` against `[[0, 0]]`	2.5
`mae` of the same	1.5
`huber` (delta 1) of prediction `[[3]]` against `[[0]]`	2.5
`lr_at("cosine", 0.1, 5, 10)`	0.05
`lr_at("cosine", 0.1, 10, 10)`	0
`lr_at("linear", 0.1, 5, 10)`	0.05
`lr_at("exp:0.9", 1.0, 2, 10)`	0.81
`lr_at("step:3:0.5", 0.1, 7, 10)`	0.025
`confusion_matrix([0, 1, 1, 0], [0, 1, 0, 0], 2)` (predictions, then targets)	[[2, 1], [0, 1]]
`prf` accuracy for those predictions	0.75
`prf` macro F1 for those predictions	0.733333
Soft update `copy_weights(a, b, 0.25)` with `a` weight 0 and `b` weight 1: new weight of `a`	0.25

10.5 Training results

Problem	Answer
XOR (section 5.2, seed 42): `predict_class(net, X)`	[0, 1, 1, 0]
XOR `gradcheck` number of checked values	41
XOR `fit` metric name	acc
Saved and reloaded XOR model: `allclose` of old and new outputs	true

10.6 Reinforcement learning

Problem	Answer
DQN target for `r = 0.5`, `gamma = 0.9`, `d = 0`, `Q_target(s') = [5, 2, 4]`	5.0
Double DQN target for the same, with `Q_online(s') = [1, 3, 2]`	2.3
Both targets when the transition is terminal (`d = 1`)	0.5
Double DQN target for `r = -0.04`, `gamma = 0.95`, `Q_online(s') = [0.1, 0.5, 0.2, 0.9]`, `Q_target(s') = [0.2, 0.6, 0.1, 0.4]`	0.34
DQN target for the same transition	0.53
Optimal return in the grid world of section 8.3 (6 steps, one goal)	0.8
Number of forward passes per minibatch for Double DQN, and for vanilla DQN	3 and 2
Section 8.3 program (seed 8): number of wins and greedy steps	360 and 6

10.7 Challenges (no answer given)

1. Train an XOR network with `"sgd"` and momentum 0.9 instead of Adam, and find the smallest hidden size and learning rate that reaches 100 percent accuracy.
2. Write a denoising autoencoder with `mlp([16, 4, 16])`, then freeze the encoder half and train a classifier head on its 4-dimensional code.
3. Extend the Double DQN program with a hard target update every 50 steps and compare the learning curve (wins per 100 episodes) with the soft update.
4. Change the XOR network of section 5.2 to end without the final sigmoid and train it with `"bce_logits"`. Make `predict_class` still return the same classes.
5. Implement a dueling Q-network by building a shared trunk plus two heads with `nn.sequential`, and combine value and advantage with tensor functions.